

Proofs for Dynamic Programming

CMSC 27200: Theory of Algorithms

1 Overview

As discussed in the lecture notes on dynamic programming, in order to give a dynamic programming algorithm we need to specify the following:

1. The **subproblems** which we are solving.
2. A **recurrence relation** for obtaining the solution to the current subproblem from solutions to smaller subproblem(s). We also need the base case(s) for this recurrence relation.

To prove that a dynamic programming algorithm is correct, we need to prove that the recurrence relation is correct. Once this is done, it is easy to show by induction that the program correctly solves all of the subproblems.

To prove the recurrence relation is correct, we need to show that $LHS \geq RHS$ and show that $RHS \geq LHS$. This is generally done through the following observations.

1. Solutions to smaller subproblems can be extended to obtain a solution to the current subproblem.
2. Solutions to the current subproblem are obtained by making a choice for the current step and then solving the resulting smaller subproblems.

Remark 1.1. *It should be noted that there are several ways to phrase this kind of proof and some of these ways prove that $LHS \geq RHS$ and $RHS \geq LHS$ at the same time. That said, the proof relies on these observations regardless of how it is phrased.*

2 Example: Weighted Interval Scheduling

For the weighted interval scheduling problem where job i has an interval $[a_i, b_i]$ and a weight w_i , recall that we let $t_1, \dots, t_{2n}$ be the start and end times of the jobs sorted from earliest to latest (we assume these times are distinct for simplicity) and we take $w(t)$ to be the maximum total weight of jobs which can be completed by time t .

Lemma 2.1. *Assuming that all the elements of $\{a_i : i \in [n]\} \cup \{b_i : i \in [n]\}$ are distinct, for all $k \in [2n - 1]$,*

1. If $t_{k+1} = a_i$ for some $i \in [n]$ then $w(t_{k+1}) = w(t_k)$.
2. If $t_{k+1} = b_j$ for some $j \in [n]$ then $w(t_{k+1}) = \max \{w(t_k), w(a_j) + w_j\}$.

Proof. Observe that if $t_{k+1} = a_i$ for some $i \in [n]$ then any schedule which finishes by time t_{k+1} also finishes by time t_k so $w(t_{k+1}) = w(t_k)$.

If $t_{k+1} = b_j$ for some $j \in [n]$, the equality $w(t_{k+1}) = \max \{w(t_k), w(a_j) + w_j\}$ can be proved by either by proving both an upper bound and a lower bound on $w(t_{k+1})$ or by reasoning about the sets of possible solutions to the subproblems.

Proof by showing both an upper and lower bound:

If $t_{k+1} = b_j$ then we can show that $w(t_{k+1}) = \max \{w(t_k), w(a_j) + w_j\}$ by showing that $w(t_{k+1}) \geq \max \{w(t_k), w(a_j) + w_j\}$ and by showing that $w(t_{k+1}) \leq \max \{w(t_k), w(a_j) + w_j\}$.

To show that $w(t_{k+1}) \geq \max \{w(t_k), w(a_j) + w_j\}$, observe that **by the definition of $w(t_k)$ there is a schedule S that finishes by time t_k with total weight $w(t_k)$** . This schedule S finishes by time t_{k+1} so we have that $w(t_{k+1}) \geq w(t_k)$. Similarly, **by the definition of $w(a_j)$, there is a schedule S which finishes by time a_j with total weight $w(a_j)$. Adding job j to this schedule gives a schedule S' which finishes by time $t_{k+1} = b_j$ and has total weight $w(a_j) + w_j$** so we have that $w(t_{k+1}) \geq w(a_j) + w_j$. Putting these pieces together, we have that $w(t_{k+1}) \geq \max \{w(t_k), w(a_j) + w_j\}$, as needed.

To show that $w(t_{k+1}) \leq \max \{w(t_k), w(a_j) + w_j\}$, observe that **by the definition of $w(t_{k+1})$ there is a schedule S' which has weight $w(t_{k+1})$ and finishes by time t_{k+1}** . There are two cases to consider. **If S' does not contain job j then S' finishes by time t_k and thus has weight at most $w(t_k)$** which implies that $w(t_{k+1}) = w(S') \leq w(t_k)$. **If S' contains job j then S' consists of a schedule S which finishes by time a_j plus the job j . $w(S) \leq w(a_j)$ so we have that $w(t_{k+1}) = w(S') = w(S) + w_j \leq w(a_j) + w_j$.** In both cases, $w(t_{k+1}) \leq \max \{w(t_k), w(a_j) + w_j\}$, as needed.

Proof which manipulates the set of possible solutions:

Let $\mathcal{S}_t$ be the set of schedules which finishes by time t . Note that $w(t) = \max_{S=(i_1, \dots, i_{|S|}) \in \mathcal{S}_t} \sum_{j=1}^{|S|} w_{i_j}$

If $t_{k+1} = b_j$ for some $j \in [n]$ then

$$\begin{aligned} \mathcal{S}_{t_{k+1}} &= \mathcal{S}_{t_k} \cup \{S' = (i_1, \dots, i_m) \in \mathcal{S}_{t_{k+1}} : i_m = j\} \\ &= \mathcal{S}_{t_k} \cup \{S' = (i_1, \dots, i_m) : i_m = j, S = (i_1, \dots, i_{m-1}) \in \mathcal{S}_{a_j}\} \end{aligned}$$

This implies that

$$\max_{S' \in \mathcal{S}_{t_{k+1}}} \{w(S')\} = \max \left\{ \max_{S' \in \mathcal{S}_{t_k}} \{w(S')\}, \max_{S \in \mathcal{S}_{a_j}} \{w(S)\} + w_j \right\}$$

and thus $w(t_{k+1}) = \max \{w(t_k), w(a_j) + w_j\}$.

Another way to phrase this proof which uses more words and less notation is as follows.

We can partition the schedules S which finish by time $t = b_j$ based on whether they contain the job j or not. Observe that

1. The schedules S which finish by time $t_{k+1} = b_j$ and do not contain job j are precisely the schedules S which finish by time t_k . The maximum weight completed by such a schedule is $w(t_k)$.
2. The schedules S which finish by time $t_{k+1} = b_j$ and contain job j are precisely the schedules of the form $S' \cup \{j\}$ where S' is a schedule which finishes by time a_j . The maximum weight completed by such a schedule is $w(a_j) + w_j$.

Thus, the maximum weight that can be completed by time $t = b_j$ is $\max \{w(t_k), w(a_j) + w_j\}$ $\square$

Remark 2.2. While this argument proves that $LHS \geq RHS$ and $RHS \geq LHS$ at the same time, this argument still uses the key observations that

1. A schedule ending by time t_k also ends by time t_{k+1} .
2. If we take a schedule ending by time a_i and then add the job j , this gives a schedule ending by time $t_{k+1} = b_j$.
3. Any schedule ending by time $t_{k+1} = b_j$ either ends by time t_k or consists of a schedule ending by time a_j plus the job j .

3 Example: The Bellman-Ford Algorithm for Shortest Paths

For the shortest path problem with negative edge weights but no negative cycles, recall that $d_k(v)$ is the minimum length of a walk from s to v which uses at most k edges.

The recurrence relation for $d_k(v)$ is as follows:

Lemma 3.1. $d_{k+1}(v) = \min \{d_k(v), \min_{u \in V(G): (u,v) \in E(G)} \{d_k(u) + l_{uv}\}\}$

Proof. Proof by showing $LHS \leq RHS$ and $RHS \leq LHS$:

To see that $d_{k+1}(v) \leq \min \{d_k(v), \min_{u \in V(G): (u,v) \in E(G)} \{d_k(u) + l_{uv}\}\}$, we make the following observations:

1. By definition, there is a walk W of length $d_k(v)$ from s to v which uses at most k edges and thus uses at most $k + 1$ edges, so $d_{k+1}(v) \leq d_k(v)$
2. By definition, for all $u \in V(G)$ there is a walk W of length $d_k(u)$ from s to u which uses at most k edges. Adding the edge (u, v) to this walk we obtain a walk from s to v of length $d_k(u) + l_{uv}$ which uses at most $k + 1$ edges.

Conversely, to see that $d_{k+1}(v) \geq \min \{d_k(v), \min_{u \in V(G): (u,v) \in E(G)} \{d_k(u) + l_{uv}\}\}$, let W be a walk from s to v of minimum length which uses at most $k + 1$ edges. We now have the following cases:

1. If W uses at most k edges then W has length at least $d_k(v)$ and thus $d_{k+1}(v) \geq d_k(v) \geq \min \{d_k(v), \min_{u \in V(G): (u,v) \in E(G)} \{d_k(u) + l_{uv}\}\}$.
2. If W uses exactly $k + 1$ edges and the final edge of W is (u, v) then W has length at least $d_k(u) + l_{uv}$ and thus $d_{k+1}(v) \geq d_k(u) + l_{uv} \geq \min \{d_k(v), \min_{u \in V(G): (u,v) \in E(G)} \{d_k(u) + l_{uv}\}\}$

We can also prove this lemma by reasoning about sets of walks.

Definition 3.2. Given $k \in \mathbb{N}$ and $v \in [n]$, define $\mathcal{W}_k(v)$ to be the set of walks from s to v using at most k edges. Note that $d_k(v) = \min_{W \in \mathcal{W}_k(v)} \{\text{length}(W)\}$ where $\text{length}(W)$ is the total length of W (i.e., the sum of the lengths of the edges in W).

Proposition 3.3. For all $k \in \mathbb{N} \cup \{0\}$,

$$\mathcal{W}_{k+1}(v) = \mathcal{W}_k(v) \cup \left(\bigcup_{u: (u,v) \in E(G)} \{W \circ (u,v) : W \in \mathcal{W}_k(u)\} \right)$$

where $W \circ (u,v)$ is the composition of the walk W and the edge (u,v) , i.e., the walk where we first take the walk W and then take the edge (u,v) .

Proof. Every walk W from s to v using at most $k+1$ edges either uses at most k edges or consists of a walk W' from s to some vertex $u \in V(G)$ using at most k edges plus the edge (u,v) . $\square$

This implies that $d_{k+1}(v) = \min \{d_k(v), \min_{u \in V(G): (u,v) \in E(G)} \{d_k(u) + l_{uv}\}\}$.

Another way to phrase this proof which uses more words and less notation is as follows.

We can partition the walks W from s to v using at most $k+1$ edges based on the vertex right before v . Observe that any walk W from s to v using at most $k+1$ edges must satisfy at least one of the following.

1. W uses at most k edges. The shortest such walk has length $d_k(v)$.
2. The vertex before v in W is u for some vertex u such that $(u,v) \in E(G)$. These walks are precisely the walks which consist of a walk W' from s to u using at most k edges and the edge (u,v) added at the end. The shortest such walk has length $d_k(u) + l_{uv}$.

Thus, the shortest walk from s to v using at most $k+1$ edges has length

$$d_{k+1}(v) = \min \{d_k(v), \min_{u \in V(G): (u,v) \in E(G)} \{d_k(u) + l_{uv}\}\}.$$

$\square$